BUET CSE 207 • EXAM EDITION

Minimum Spanning Trees & Single-Source Shortest Paths

A concept-first, proof-aware, dry-run-heavy guide for a 2–3 hour complete revision.

Inside: graph foundations • generic MST • Kruskal • Prim • second-best MST • relaxation • Dijkstra • Bellman–Ford • DAG shortest paths • difference constraints • negative cycles • formal properties and proofs • exam/viva bank • five-page final revision

Prepared from the scope of CLRS, 3rd ed., Chapters 23–24 • Original diagrams and exam-focused explanations

How to use this note

FIRST 60 MIN
Read Graph Basics, MST properties, Kruskal and Prim. Redraw one dry run.

NEXT 60 MIN
Master relaxation, Dijkstra, Bellman–Ford and DAG-SP. Trace the tables.

FINAL 30 MIN
Attempt the viva/theory prompts, then use the five revision pages only.

NOTATION
V = vertices, **E** = edges, **w(u,v)** = edge weight, **δ(s,v)** = true shortest-path weight, **d[v]** = current estimate, **π[v]** = parent/predecessor. "∞" means not yet reachable.

Contents

One picture: do not confuse the two problems

Problem	What is minimized?	Output	Typical algorithms
MST	Total weight of a tree connecting *all* vertices	V−1 undirected edges; no designated source	Kruskal, Prim
SSSP	Path weight from one source s to each reachable vertex	Shortest-path tree/forest of predecessor edges	Dijkstra, Bellman–Ford, DAG-SP

Classic trap: an MST path between two vertices need not be their shortest path. MST minimizes the *sum of selected tree edges*, not every pairwise route.

1. Graph basics — the minimum you must own

Term	Exam-ready definition	Key consequence
Graph	G=(V,E), vertices plus edges connecting pairs.	Edges are ordered pairs in directed graphs; unordered in undirected.
Weighted graph	Each edge e has numerical weight w(e).	Weight can mean cost, time, distance, loss; it may be negative in directed SSSP.
Walk / path	A walk follows adjacent edges; a simple path repeats no vertex.	A shortest path can be chosen simple if no reachable negative cycle affects it.
Cycle	A closed path with first=last; simple cycle repeats no other vertex.	A tree has no cycle.
Connected	Every pair of vertices has a path (undirected case).	A spanning tree exists iff the graph is connected.
Component	A maximal connected subgraph.	Disconnected graph → minimum spanning *forest*.
Tree / forest	Connected acyclic undirected graph / disjoint union of trees.	A tree on |V| vertices has exactly |V|−1 edges.
Spanning tree	A subgraph that contains all vertices and is a tree.	Remove any edge → disconnected; add any non-tree edge → one cycle.

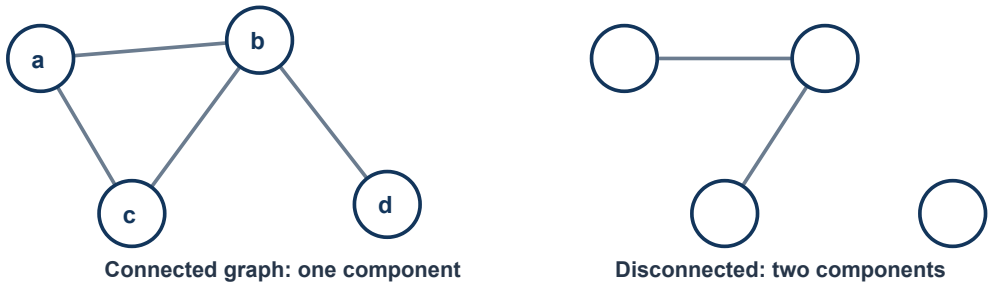

Figure 1.1 — Connectivity and connected components.

Representations

Representation	Space	Edge test	Enumerate neighbors	Use when
Adjacency matrix	$\Theta(V^2)$	$\Theta(1)$	$\Theta(V)$	Dense graph; fast edge lookup; simple Prim array implementation.
Adjacency list	$\Theta(V+E)$	Usually O(deg u)	$\Theta(\deg u)$	Sparse graph; Dijkstra/Prim/BFS/DFS.
Edge list	$\Theta(E)$	$\Theta(E)$	$\Theta(E)$	Kruskal and Bellman–Ford: scan every edge.

> **Memory trick:** **M**atrix = **M**any cells; **L**ist = only **L**ive edges; **E**dge list = algorithms that repeatedly scan **E**.

2. Minimum Spanning Tree: meaning and structure

INTUITION

You must connect every campus building with minimum total cable. Redundant loops waste cable, so an optimum connected solution is a tree.

Formal definition. For a connected, undirected, weighted graph G=(V,E), an MST is a spanning tree T⊆E minimizing $w(T)=\Sigma_{e\in T}w(e)$. It has exactly |V|−1 edges.

Applications

- Least-cost roads, fiber, power, pipelines
- Clustering: delete expensive MST edges
- Network broadcast backbones
- Approximation algorithms and image segmentation

Important boundaries

- MST is normally defined on undirected graphs.
- Negative weights are completely safe.
- Disconnected input produces an MSF, not one MST.
- Weights need not represent geometric distance.

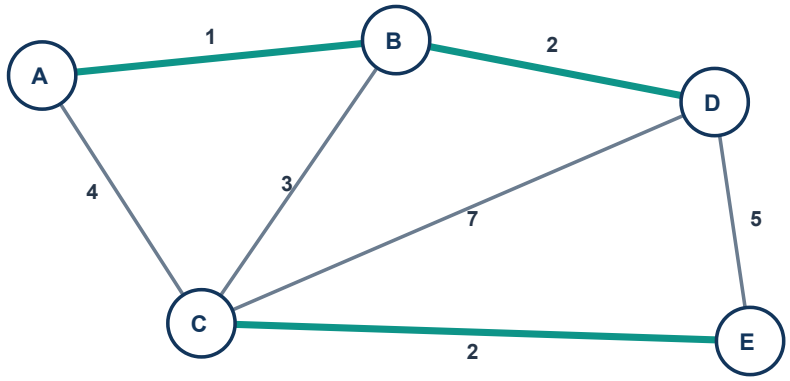

Figure 2.1 — Green edges AB(1), BD(2), CE(2), plus BC(3) form an MST of weight 8.

Uniqueness

- If **all edge weights are distinct**, the MST is unique (sufficient, not necessary).
- Equal weights may permit multiple MSTs, but do not guarantee it.
- All MSTs have the same total weight, even when their edge sets differ.

Fast uniqueness test: in Kruskal, if at some weight level different non-cycling choices can connect the same current components, alternative MSTs may exist. Formal testing groups equal-weight edges before union.

3. The greedy foundation: cut, light, safe, cycle

Cut property

A **cut** (S,V−S) partitions the vertices. An edge **crosses** the cut if its endpoints lie on opposite sides. A **light edge** is a minimum-weight crossing edge.

> **Cut property.** Let A be a subset of some MST. If a cut respects A (no edge of A crosses it), then any light edge crossing that cut is **safe** for A: adding it preserves the possibility of completing A to an MST.

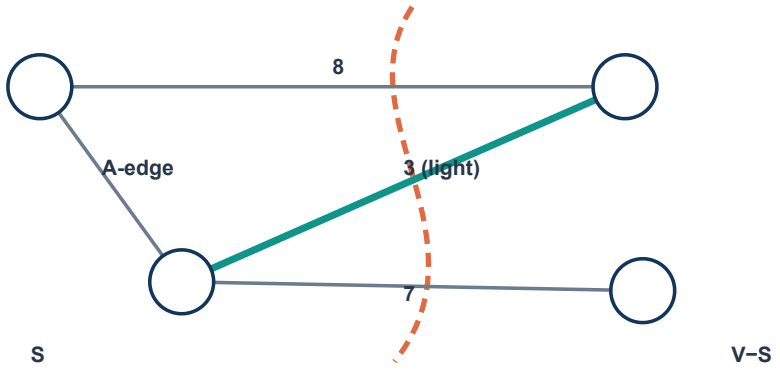

Figure 3.1 — Dashed line is a cut respecting A. Weight-3 crossing edge is light and therefore safe.

Why the safe edge works: exchange argument

1. Take an MST T that contains current edges A.
2. If T already contains light edge (u,v), done.
3. Otherwise adding (u,v) to T creates exactly one cycle.
4. That cycle crosses the cut again through some edge (x,y).
5. Because (u,v) is light, $w(u,v) \le w(x,y)$. Replace (x,y) by (u,v); the result is still a spanning tree and no heavier. Hence it is also an MST containing $A \cup \{(u,v)\}$.

> **Light edge** is relative to a particular cut: minimum among crossing edges.

> **Safe edge** is relative to current partial solution A: it can be added without losing all MST completions.

The CLRS generic MST framework

```
GENERIC-MST(G,w)
A ← ∅
while A does not form a spanning tree:
    find an edge (u,v) that is safe for A
    A ← A ∪ {(u,v)}
return A
```

Loop invariant: before every iteration, A is contained in some MST. Initially $A=\emptyset$. A safe edge preserves the invariant; after V−1 additions A itself is an MST. Kruskal and Prim differ only in how they choose a safe edge.

Cycle property

> **Cycle property.** If an edge is strictly heavier than every other edge on a cycle, it belongs to no MST. More generally, there exists an MST excluding any maximum-weight cycle edge.

Intuition: delete the heaviest edge; the remaining cycle path still connects its endpoints and is no more expensive.

Wording trap: with tied maximum weights, do not say “the edge is in no MST.” Say “there is an MST that excludes it.” Strictly unique heaviest gives the stronger claim.

4. Kruskal's algorithm

CORE IDEA

Build a forest. Consider edges globally from lightest to heaviest; accept an edge exactly when it joins two different components.

```
KRUSKAL(G,w)
A ← ∅
for each vertex v ∈ G.V: MAKE-SET(v)
sort G.E into nondecreasing order by weight
for each edge (u,v) in sorted order:
    if FIND-SET(u) ≠ FIND-SET(v):
        A ← A ∪ {(u,v)}
        UNION(u,v)
return A
```

Dry run on Figure 2.1

Order	Edge	Action	Forest components after action
1	AB(1)	Take	{AB}{C}{D}{E}
2	BD(2)	Take	{ABD}{C}{E}
3	CE(2)	Take	{ABD}{CE}
4	BC(3)	Take	{ABCDE}; now V−1 edges, stop
5+	AC(4), DE(5), CD(7)	Would reject	Endpoints already connected

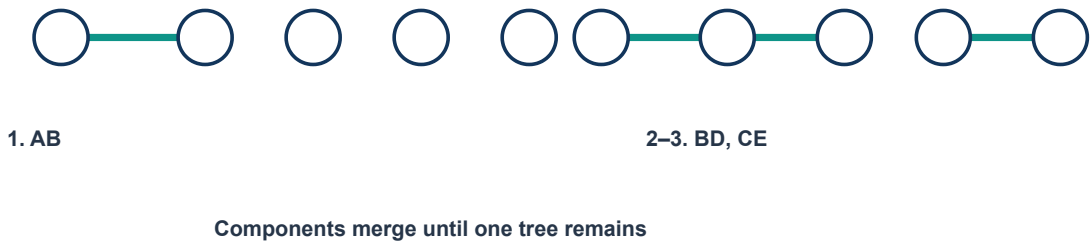

Figure 4.1 — Union–Find tracks components; it does not store the final MST shape.

Why it works

Before accepting (u,v), its endpoints lie in two components. The cut "vertices in u's component versus everything else" respects A. Since all lighter candidates have already been processed, (u,v) is a light crossing edge; by the cut property it is safe.

Complexity

$$\text{sorting: } O(E \lg E) = O(E \lg V) \quad + \quad \text{DSU: } O(E\,\alpha(V)) \quad \Rightarrow \quad \mathbf{O(E \lg V)}$$

Space: O(V) auxiliary for DSU plus O(E) to store/sort edges. α is inverse Ackermann, effectively constant.

5. Disjoint Set Union (Union–Find)

Operations

- **MAKE-SET(x):** singleton component.
- **FIND-SET(x):** return representative/root.
- **UNION(x,y):** merge roots if different.

Two optimizations

- **Union by rank/size:** attach shorter tree below taller.
- **Path compression:** during FIND, point visited nodes directly to root.

```
FIND(x)
if parent[x] ≠ x: parent[x] ← FIND(parent[x])   // compression
return parent[x]

UNION(x,y)
rx ← FIND(x); ry ← FIND(y)
if rx = ry: return
if rank[rx] < rank[ry]: parent[rx] ← ry
else: parent[ry] ← rx
      if rank[rx] = rank[ry]: rank[rx]++
```

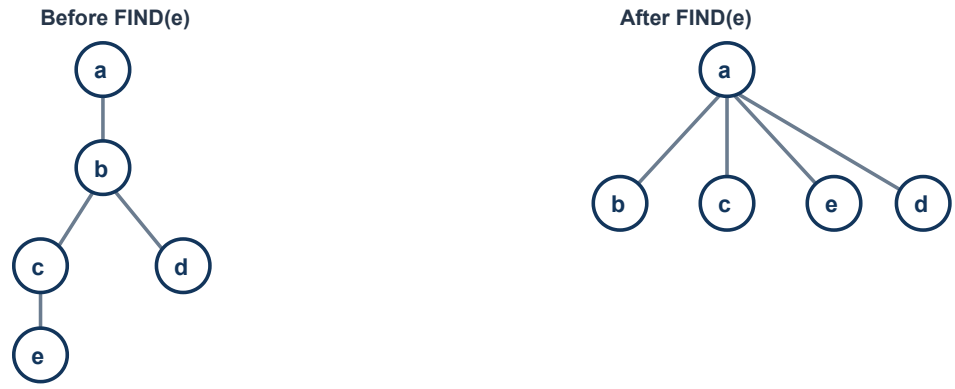

Figure 5.1 — Path compression flattens the route e→c→b→a to e→a (and also c→a).

Amortized language matters: m operations with both optimizations take O(m α(V)), not necessarily constant worst-case per individual operation.

Kruskal: strengths, weaknesses, traps

Strengths	Weaknesses	Common mistakes
Excellent for sparse graphs; natural edge list; easily gives MSF; simple correctness proof.	Must sort all edges; less natural when graph arrives only as dense matrix.	Using DFS cycle test each time; unioning vertices rather than roots; stopping before V−1 accepted edges; rejecting equal weights automatically; applying to directed graph.

6. Prim's algorithm

> **CORE IDEA**
> Grow one tree from an arbitrary root. At every step, attach the outside vertex with the cheapest edge into the current tree.

key[v] = cheapest known crossing edge from the current tree to v. **parent[v]** stores that edge's tree endpoint. The min-priority queue contains vertices not yet finalized.

```
PRIM(G,w,r)
for each u ∈ V: key[u] ← ∞; parent[u] ← NIL
key[r] ← 0; Q ← V
while Q ≠ ∅:
    u ← EXTRACT-MIN(Q)
    for each v ∈ Adj[u]:
        if v ∈ Q and w(u,v) < key[v]:
            parent[v] ← u
            key[v] ← w(u,v)
            DECREASE-KEY(Q,v,key[v])
return {(parent[v],v) : v ≠ r}
```

Dry run (start A) on Figure 2.1

Extract	Relax/update keys	Chosen edge	Keys after step (unseen only)
A:0	B←1, C←4	—	B1, C4, D∞, E∞
B:1	C←3, D←2	AB	D2, C3, E∞
D:2	E←5; C stays 3	BD	C3, E5
C:3	E←2	BC	E2
E:2	—	CE	done; total 8

Figure 6.1 — Prim maintains a frontier around one growing tree.

Why it works

At every iteration, the cut is "vertices already extracted" versus "vertices still in Q." It respects chosen tree edges. The minimum key is a light edge crossing this cut, so the cut property makes it safe.

7. Prim implementations and Kruskal comparison

Prim implementation	Extract-min	Decrease-key/update	Total	Best fit
Adjacency matrix + array	O(V), repeated V	O(1)	**$O(V^2)$**	Dense graph
Adjacency list + binary heap	O(lg V)	O(lg V), up to E	**O(E lg V)**	Sparse graph, practical default
Adjacency list + Fibonacci heap	O(lg V) amortized	O(1) amortized	**O(E+V lg V)**	Theoretical bound

Priority-queue trap: Prim's priority is the cheapest *single connecting edge* key[v], not the accumulated root-to-v distance. Accumulated distance is Dijkstra.

Kruskal vs Prim

Feature	Kruskal	Prim
Growth	Many components merge (forest)	One connected tree grows
Greedy choice	Globally lightest edge joining different components	Lightest edge leaving current tree
Primary structure	Sorted edge list + DSU	Adjacency structure + min-priority queue
Typical time	O(E lg V)	O(E lg V) heap; $O(V^2)$ array
Sparse/dense	Usually excellent sparse	Heap sparse; matrix/array excellent dense
Disconnected input	Naturally returns MSF	Restart per component for MSF
Sorting	Required	Not required
Memory emphasis	All edges + O(V) DSU	Adjacency + O(V) keys/parents/heap

Prim common mistakes

- Updating an already extracted vertex; forgetting the $v \in Q$ test.
- Adding the root's NIL parent as an edge.
- Summing keys before they are final; sum chosen/final key values only.
- Using a max-heap, or failing to implement decrease-key (lazy duplicates are fine if stale entries are skipped).
- Claiming the starting root changes optimal weight; it may change which MST appears under ties, never the optimum value.

Maximum-mark answer skeleton: (1) one-line idea, (2) pseudocode, (3) invariant/cut property, (4) complexity by data structure, (5) one dry-run table. This structure works for both MST algorithms.

8. Second-best MST

Definition. A spanning tree whose total weight is the smallest value strictly greater than the MST weight. Some courses use “different from T” even if equal; state your convention.

> **KEY EXCHANGE IDEA**
>
> Fix an MST T. Add a non-tree edge e=(u,v). It creates one cycle. Delete one edge f on the unique u–v path in T. To obtain the cheapest tree containing e, delete the maximum-weight edge on that path.

$$\text{candidate}(e) = w(T) + w(e) - \text{maxWeightOnPath}_T(u,v)$$

```
SECOND-BEST-MST(G)
T ← MST(G); best ← ∞
for each non-tree edge e=(u,v):
    f ← maximum-weight edge on unique path u→v in T
    candidate ← w(T) + w(e) − w(f)
    if candidate > w(T): best ← min(best,candidate) // strict convention
return best and corresponding exchange (e,f)
```

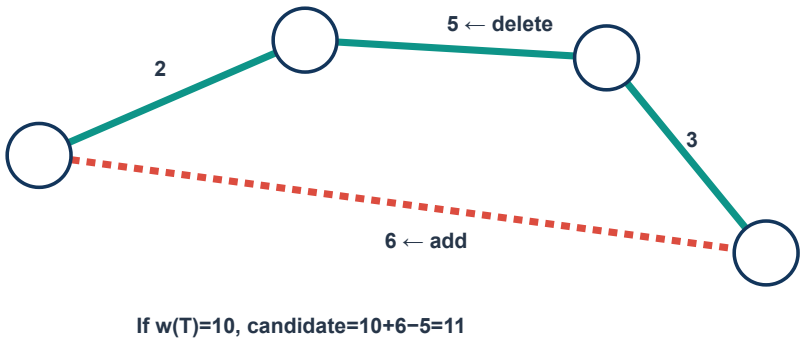

Figure 8.1 — Add a non-tree edge, then remove the heaviest tree edge on the resulting cycle.

Complexity

- Simple: for each of O(E) non-tree edges, DFS the tree in O(V): **O(EV)** after MST.
- Advanced: preprocess binary lifting/LCA for maximum-on-path queries in O(V lg V), then O(lg V) each: **O(E lg V)** overall.

> **Ties:** If candidate equals w(T), you found another MST, not a strictly second-best tree. For the strict version, find the smallest positive increase; tied path maxima may require careful alternate-edge handling.

9. SSSP foundation: paths, estimates, relaxation

> **INTUITION**
> Place a tentative price tag d[v] on each vertex. Relaxation asks: "Would reaching v through u be cheaper than my current tag?" If yes, lower the tag and remember u.

For directed weighted G=(V,E), source s, the true shortest-path value is:

$$\delta(s,v) = \min\{\, w(p) : p \text{ is a path } s \to v \,\}; \quad \delta(s,v)=\infty \text{ if unreachable}$$

If a reachable negative cycle can reach v, path weights can decrease without bound; conventionally $\delta(s,v)=-\infty$ and no finite shortest path exists.

```
INITIALIZE-SINGLE-SOURCE(G,s)
for each v ∈ V: d[v] ← ∞; parent[v] ← NIL
d[s] ← 0

RELAX(u,v,w)
if d[v] > d[u] + w(u,v):
    d[v] ← d[u] + w(u,v)
    parent[v] ← u
```

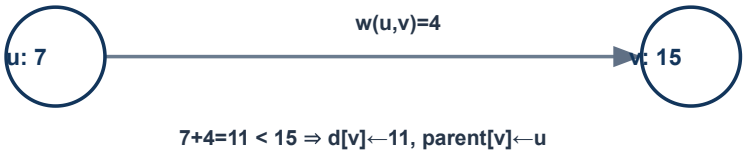

Figure 9.1 — One relaxation improves an upper bound; it need not finalize it.

Path reconstruction

Follow parent pointers backward v→parent[v]→…→s, then reverse. If d[v]=∞, v is unreachable. The parent subgraph is a shortest-path tree only after the algorithm's correctness conditions hold.

Core properties worth quoting

- **Upper-bound:** always $d[v] \geq \delta(s,v)$; relaxation never invents a path.
- **Triangle inequality:** $\delta(s,v) \leq \delta(s,u)+w(u,v)$.
- **Convergence:** if s→u is already correct and (u,v) is the next edge of a shortest path, relaxing it makes v correct.
- **Path relaxation:** relaxing edges of a shortest path in order makes its endpoint correct.

10. Dijkstra’s algorithm

WHEN VALID

All edge weights reachable from the source must be nonnegative. Then the unsettled vertex with smallest estimate can never later receive a cheaper route.

```
DIJKSTRA(G,w,s)
initialize d and parent; d[s] ← 0
Q ← min-priority queue containing (0,s)
while Q not empty:
    (du,u) ← EXTRACT-MIN(Q)
    if du ≠ d[u]: continue              // skip stale lazy entry
    for each edge (u,v):
        if d[v] > d[u] + w(u,v):
            d[v] ← d[u] + w(u,v)
            parent[v] ← u
            push (d[v],v) into Q
```

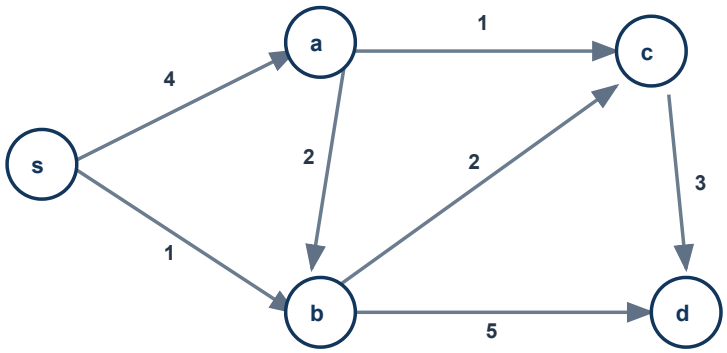

Figure 10.1 — Dijkstra example. Directed edges: s→a4, s→b1, b→a2, a→c1, b→c2, b→d5, c→d3.

Settled	Updates	d[s,a,b,c,d]	Queue snapshot
s(0)	a=4, b=1	[0,4,1,∞,∞]	(1,b),(4,a)
b(1)	a=3,c=3,d=6	[0,3,1,3,6]	(3,a),(3,c),(4,a stale),(6,d)
a(3)	c stays 3	[0,3,1,3,6]	(3,c),(4 stale),(6,d)
c(3)	d stays 6 (tie)	[0,3,1,3,6]	(6,d)
d(6)	—	final	empty

11. Dijkstra: proof, complexity, and traps

Proof intuition — first wrong vertex contradiction

1. Assume u is the first extracted vertex whose estimate is not the true shortest distance.
2. On a shortest s→u path, let y be the first not-yet-extracted vertex; x immediately before it is settled.
3. When x was processed, relaxation ensured d[y]=δ(s,y) (or no larger).
4. With nonnegative remaining edges, δ(s,y)≤δ(s,u).
5. Because u had minimum queue key, d[u]≤d[y]≤δ(s,u), while upper-bound gives d[u]≥δ(s,u). Thus equality—a contradiction.

> **The load-bearing assumption:** nonnegative weights ensure a path cannot get cheaper after y. Without it, the greedy "finalize minimum" step breaks.

Complexity

Implementation	Time	Notes
Array + adjacency matrix	$O(V^2)$	Good dense graph; scan to find minimum.
Binary heap + adjacency list	O((V+E)lg V), usually O(E lg V)	Practical sparse default.
Fibonacci heap	O(E+V lg V) amortized	Theoretical improvement.

Space: O(V+E) graph storage and O(V) logical state; lazy-duplicate heaps may temporarily hold O(E) entries.

Failure with a negative edge

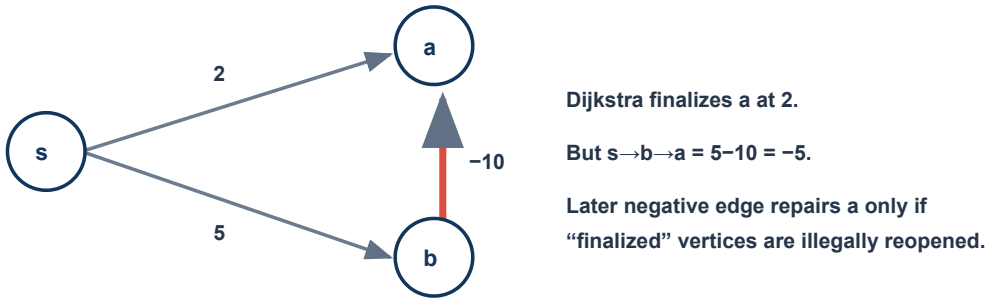

Figure 11.1 — A single negative edge can invalidate greedy finalization even without a negative cycle.

> **Common mistakes:** using Dijkstra merely because there is no negative cycle; marking visited when inserted rather than extracted; confusing BFS (only unweighted/equal weights); omitting stale-entry handling; assuming negative edges in unreachable components matter (they do not affect this source).

12. Bellman–Ford

> **CORE IDEA**
> Do not greedily finalize. Repeatedly relax every edge. After i full passes, all shortest paths using at most i edges are correctly represented.

```
BELLMAN-FORD(G,w,s)
initialize d and parent; d[s] ← 0
for i ← 1 to |V|−1:
    changed ← false
    for each edge (u,v) ∈ E:
        if d[u] ≠ ∞ and d[v] > d[u] + w(u,v):
            d[v] ← d[u] + w(u,v); parent[v] ← u
            changed ← true
    if not changed: break
for each edge (u,v) ∈ E:
    if d[u] ≠ ∞ and d[v] > d[u] + w(u,v):
        report a reachable negative cycle
return d,parent
```

Why V−1 passes?

With no reachable negative cycle, a shortest path can be simple. A simple path visits at most V vertices and uses at most V−1 edges. Each full pass guarantees progress by at least one edge along such a path (regardless of edge-list order).

Pass-level dry run

Edges are deliberately ordered poorly: (a,b,−2), (s,a,4), (b,c,3), (s,c,10).

Pass	d[s,a,b,c] after full scan	Meaning
0	[0,∞,∞,∞]	Initialization
1	[0,4,∞,10]	s→a and s→c known; a→b was scanned too early
2	[0,4,2,5]	a→b then b→c propagate improvements
3	[0,4,2,5]	No change: early stop; no reachable negative cycle

> **In-place nuance:** one pass can propagate through multiple edges if the order is favorable. The theorem promises "at least paths of i edges after i passes," not "exactly one edge per pass."

Complexity and uses

Time **O(VE)**; space **O(V)** beyond graph storage. Use for reachable negative edges, negative-cycle detection, distance-vector routing, difference constraints, and as a correctness oracle for small tests.

13. Negative cycles: detection and meaning

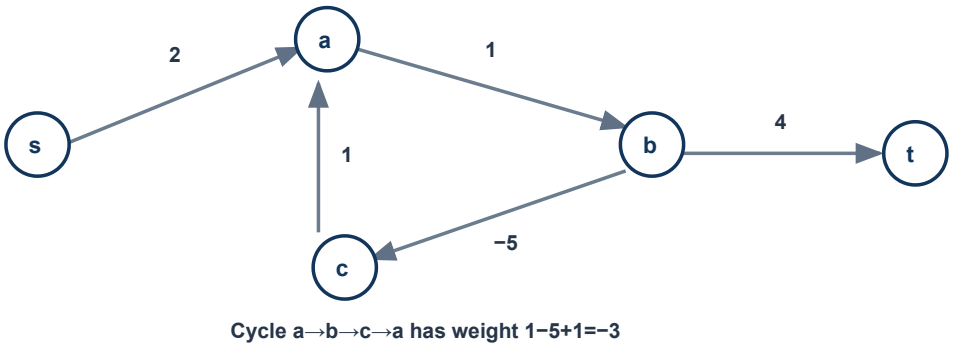

Figure 13.1 — Each loop reduces cost by 3; then going to t gives arbitrarily small s→t walks.

Why no shortest path exists

If source s can reach a negative cycle C and C can reach v, loop around C k times before proceeding to v. Cost decreases by k|w(C)|, so there is no finite minimum.

> **Bellman–Ford detection:** after V−1 passes, any still-relaxable edge reachable from s proves that some reachable negative cycle exists. To mark every affected vertex, collect improved vertices on pass V and graph-search forward from them.

Recover an actual cycle

1. Record a vertex x improved on the Vth pass.
2. Follow parent pointers V times; this guarantees entering a cycle.
3. From that vertex, follow parents until it repeats; reverse orientation if presenting forward edges.

> **Reachability matters:** a negative cycle in an unreachable component must not make source-based Bellman–Ford report failure. Check d[u]≠∞ before using edge (u,v), especially in finite-integer code where "INF + negative" can overflow.

Real-life interpretation

A negative cycle may represent an arbitrage loop (money grows every round), a time/cost model contradiction, or inconsistent difference constraints—not a physically negative road length.

14. Shortest paths in a DAG

WHY FASTER

A DAG has a topological order in which every edge goes forward. Process each vertex once in that order; no future edge can return to invalidate it. Negative edges are safe because a DAG has no cycle at all.

```
DAG-SHORTEST-PATHS(G,w,s)
topologically sort vertices
initialize d and parent; d[s] ← 0
for each u in topological order:
    if d[u] ≠ ∞:
        for each edge (u,v): RELAX(u,v,w)
return d,parent
```

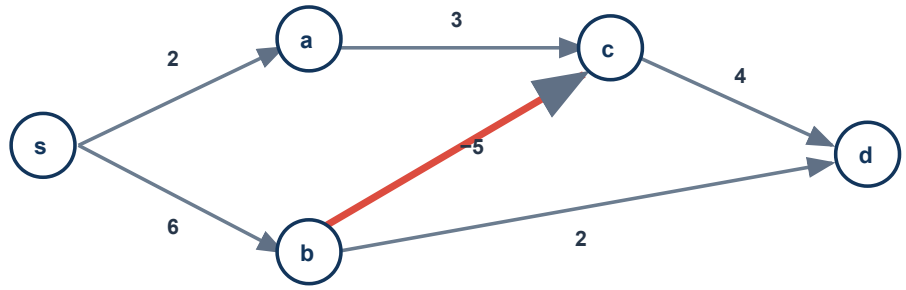

Figure 14.1 — The negative edge b→c is harmless. Distances: s=0, a=2, b=6, c=1, d=5.

Complexity

Topological sort O(V+E) plus each edge relaxed once O(E): **Θ(V+E)**; space O(V) beyond adjacency lists.

Applications

- Critical-path/project scheduling (often use longest paths by negating or changing min to max).
- Dependency networks, layered graphs, dynamic-programming state graphs.
- Fast paths in acyclic routing/workflow models.

Trap: "DAG shortest path handles negative cycles" is nonsense: DAGs contain no cycles. Also do not topologically sort a general directed graph and proceed; the order exists iff it is acyclic.

15. Master SSSP comparison

Feature	Dijkstra	Bellman–Ford	DAG-SP
Weight restriction	All reachable edges ≥0	Negative edges allowed	Any weights
Negative cycle	Not supported	Detects reachable ones	Impossible
Main strategy	Greedy finalization	Repeated global relaxation	Topological dynamic programming
Main structure	Min-PQ + adjacency list	Edge list	Topological order + adjacency list
Time	O(E lg V) binary heap; $O(V^2)$ array	O(VE)	Θ(V+E)
Best use	Road/network cost without negative weights	Negative weights, cycle test, constraints	Acyclic dependency/state graph
Main limitation	Negative edge breaks proof	Slower	Only DAGs

Decision flow

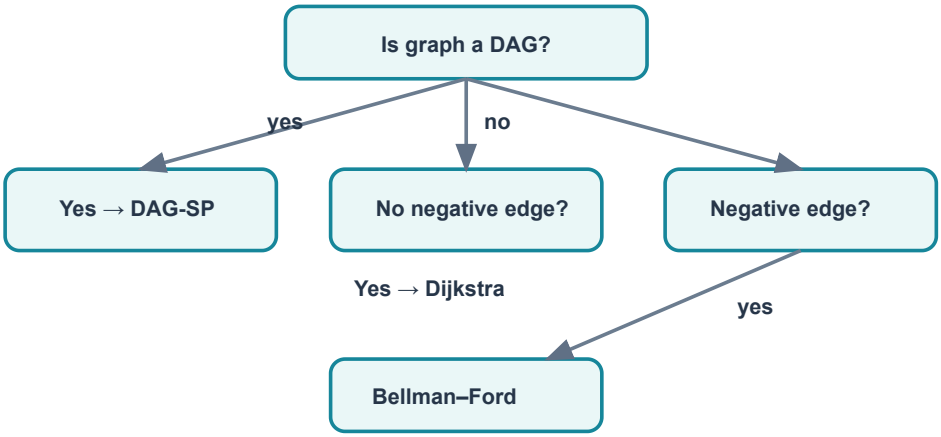

Figure 15.1 — Choose by graph structure first, then by edge signs.

Relaxation correctness in one paragraph

Every finite d[v] is the weight of some discovered s→v path, so it cannot be below the true minimum. When predecessor u is already correct and (u,v) is the next edge on a shortest path, triangle inequality plus upper-bound squeeze d[v] to δ(s,v). Algorithms differ mainly in how they guarantee the right relaxations occur in a sufficient order.

16. Complete complexity sheet

Algorithm	Time	Auxiliary space*	Key structure	Use when
Kruskal	O(E lg V)	O(V) + edge sorting	Edge list, DSU	Sparse undirected; MSF
Prim (array)	$O(V^2)$	O(V)	Matrix, arrays	Dense undirected
Prim (binary heap)	O(E lg V)	O(V)	Adjacency list, min-heap	Sparse undirected
Prim (Fibonacci)	O(E+V lg V)	O(V)	Fibonacci heap	Theoretical bound
Second MST (simple)	O(EV)	O(V)	MST + tree search	Small graph/exam method
Second MST (LCA)	O(E lg V)	O(V lg V)	Binary lifting max query	Large graph
Dijkstra (array)	$O(V^2)$	O(V)	Matrix, arrays	Dense, nonnegative
Dijkstra (binary heap)	O(E lg V)	O(V), or O(E) lazy PQ	Adjacency list, min-heap	Sparse, nonnegative
Bellman–Ford	O(VE)	O(V)	Edge list	Negative edges/cycle detection
DAG-SP	Θ(V+E)	O(V)	Topo order, adjacency	Directed acyclic graph

*Excludes storage of input graph unless stated. An adjacency list itself uses Θ(V+E); a matrix uses $\Theta(V^2)$.

Proof map

Claim	Proof tool	One-line invariant
Kruskal correct	Cut property + exchange	Accepted forest A is contained in some MST.
Prim correct	Cut property + exchange	Growing tree edges are contained in some MST.
Dijkstra correct	First wrong extraction contradiction	Every settled vertex has $d=\delta$.
Bellman–Ford correct	Induction on number of path edges	After i passes, paths with $\le i$ edges are represented.
DAG-SP correct	Induction in topological order	All incoming predecessors are processed before v.

> **Mnemonic:** MST algorithms ask "which **edge** is safe?" SSSP algorithms ask "which **estimate** becomes exact?"

17. Exam and viva preparation

Frequently asked theory questions

1. State and prove the cut property using an exchange argument.
2. Trace Kruskal, showing sorted edges and DSU components after each acceptance.
3. Trace Prim, showing key and parent arrays after every extraction.
4. Compare Prim and Kruskal for sparse and dense graphs.
5. Find a second-best MST by one-edge exchange.
6. Define relaxation and prove the path-relaxation property intuitively.
7. Prove Dijkstra and identify exactly where nonnegativity is required.
8. Give the smallest counterexample to Dijkstra with a negative edge.
9. Explain why Bellman–Ford needs $V-1$ passes and a Vth test.
10. Find/reconstruct a reachable negative cycle.
11. Why can DAG shortest path allow negative weights?

Rapid viva: question → answer

Question	Strong one-line answer
Can MST contain negative edges?	Yes; greedy cut logic remains valid and such edges are often selected.
Does distinct weight imply unique MST?	Yes. The converse is false.
Can Dijkstra handle a negative edge if no negative cycle?	No; a later negative edge can improve a finalized vertex.
Can Bellman–Ford detect every negative cycle?	Only cycles reachable from its source, unless a super-source is added.
Why $V-1$ edges in a tree?	Connected acyclic graphs have exactly $V-1$; adding one creates a cycle.
Prim key vs Dijkstra distance?	Prim key is one crossing-edge weight; Dijkstra distance is total source-path weight.
MST versus shortest-path tree?	They optimize different objectives and generally differ.
What if graph is disconnected?	No spanning tree; Kruskal gives a minimum spanning forest.

How to earn method marks in a dry run

- Write assumptions: directed/undirected, start/source, tie-breaking rule.
- Show state, not only final answer: DSU sets; key/parent; d/parent; queue.
- Circle accepted/finalized choices; cross rejected cycle edges or stale heap entries.
- Finish with selected edges/path, total weight/distances, and complexity.

18. Common traps and memory anchors

MST TRAPS

- Forgetting "undirected, connected."
- Calling every globally smallest edge safe without naming a cut.
- Using Prim's accumulated path weight.
- Stopping Kruskal after V−1 considered rather than accepted edges.
- Claiming ties always mean multiple MSTs.

SSSP TRAPS

- Using Dijkstra on any reachable negative edge.
- Relaxing from ∞ without a guard in finite arithmetic.
- Confusing an edge count with a pass count.
- Reporting unreachable negative cycles.
- Forgetting to reverse parent chain.

Mnemonics

Mnemonic	Meaning
Kruskal: Sort–Check–Union	Sort edges; check representatives; union if distinct.
Prim: Pull nearest branch	Pull outside vertex using cheapest edge into current tree.
Dijkstra: Done means done	Extraction finalizes only because weights are nonnegative.
Bellman–Ford: V−1 sweep, one more peep	Relax V−1 passes, then inspect once more for a negative cycle.
DAG: order once, relax once	Topological processing makes one edge scan sufficient.

Mini practice set

1. Prove or disprove: "The lightest edge of a connected graph belongs to every MST." (If unique globally lightest: yes; with ties, at least one MST contains a chosen light edge, but not necessarily every MST.)
2. Can an MST exclude the second-lightest edge? (Yes, e.g., it may close a cycle.)
3. How do you find an MSF with Prim? (Restart Prim at each unvisited component.)
4. What does an unchanged Bellman–Ford pass prove? (All reachable estimates are stable; stop early.)
5. For an unweighted graph, best SSSP? (BFS, $\Theta(V+E)$.)

Final distinction: A "visited" flag means different things. In Prim it means vertex already in the MST tree. In Dijkstra it means shortest distance finalized. In neither algorithm should it normally be set merely upon first discovery.

19. CLRS SSSP foundations — complete chapter scope

Four shortest-path problem variants

Variant	Task	Reduction / remark
Single source	From fixed s to every v	The focus of Chapter 24.
Single destination	From every v to fixed t	Reverse every edge, then run SSSP from t.
Single pair	From given u to given v	Run SSSP from u; known worst-case bounds are not asymptotically better in the general model.
All pairs	For every ordered pair (u,v)	Could run SSSP V times; specialized algorithms are studied in CLRS Chapter 25.

Unweighted graph: BFS is already an SSSP algorithm when every edge has unit weight; it runs in $\Theta(V+E)$.

Optimal substructure (Lemma 24.1)

Subpaths of shortest paths are shortest. If p is a shortest u→v path, every subpath x→y within p is shortest between x and y. Otherwise replace that subpath by a cheaper one and obtain a cheaper u→v path—a contradiction.

This is the structural reason greedy methods and dynamic programming can work. It does *not* say every combination of independently shortest subpaths is a valid simple path.

What cycles can occur in a shortest path?

Cycle weight	Effect	Conclusion
Positive	Removing it makes the path cheaper.	Cannot appear in a shortest path.
Zero	Removing it preserves weight.	A shortest simple path still exists.
Negative	Repeating it makes cost arbitrarily low.	No finite shortest path to vertices reachable beyond it.

Therefore, when no relevant negative cycle exists, we may choose a shortest path that is simple and uses at most **$V-1$ edges**. This fact drives Bellman–Ford's pass count.

Infinity conventions and storage

- CLRS conceptually uses $a+\infty=\infty$ and $\min(\infty,x)=x$. In code, guard d[u]≠INF and avoid overflow.
- Chapter 24 assumes weighted adjacency lists, giving $\Theta(V+E)$ graph space and $O(1)$ weight access per traversed edge.
- Bellman–Ford is often written with an edge list because every pass scans all edges.

20. Shortest-path trees and the predecessor subgraph

Algorithms must return not only distances but actual paths. Each non-source vertex v stores a predecessor π[v]. Following π backward should lead from v to s.

> **Predecessor subgraph $G_\pi=(V_\pi,E_\pi)$:**
> $V_\pi=\{s\}\cup\{v:\pi[v]\neq NIL\}$; $E_\pi=\{(\pi[v],v):v\in V_\pi-\{s\}\}$.

Formal shortest-path tree requirements

1. Its vertex set is exactly the vertices reachable from s.
2. It is a directed tree rooted at s.
3. Its unique s→v tree path is a shortest s→v path in G for every reachable v.

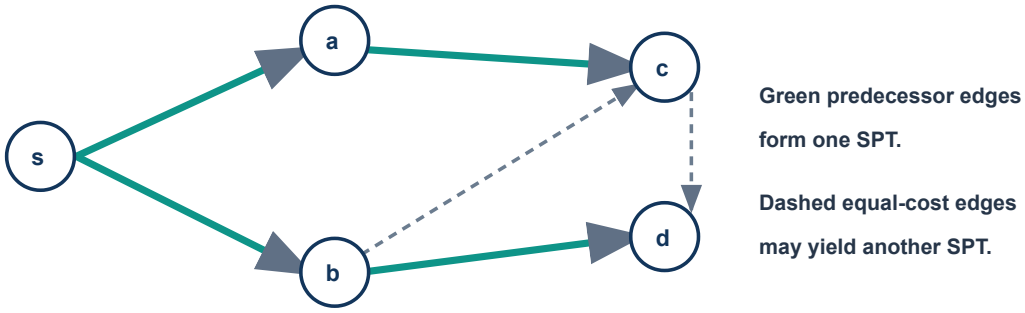

Figure 20.1 — Shortest paths and shortest-path trees need not be unique, even though all correct distance values agree.

PRINT-PATH and tie behavior

```
PRINT-PATH(s,v)
if v=s: print s
else if parent[v]=NIL: print "no path"
else: PRINT-PATH(s,parent[v]); print v
```

- A strict relaxation test (>) keeps the first equal-cost predecessor; using ≥ may change parents and, with zero cycles, needs care.
- During execution, parents may describe only provisional routes. They become an SPT when the distance-correctness and no-reachable-negative-cycle assumptions hold.
- Unreachable vertices keep d=∞ and π=NIL and do not belong to V_π (except s itself).

> **Shortest-path tree ≠ MST:** an SPT is directed outward from a source and preserves its distances. Its total edge weight need not be minimum.

21. Difference constraints and shortest paths (CLRS 24.4)

WHY THIS SECTION EXISTS

Bellman–Ford can solve a useful special case of linear programming: inequalities comparing the difference of two unknowns.

From an inequality to an edge

$$x_i - x_j \le b_k \iff x_i \le x_j + b_k \Rightarrow \text{edge } v_j \to v_i \text{ of weight } b_k$$

Build a **constraint graph** with one vertex v_i per variable and one directed weighted edge per inequality. Add a new source v_0 with zero-weight edges $v_0 \to v_i$ so every variable is reachable.

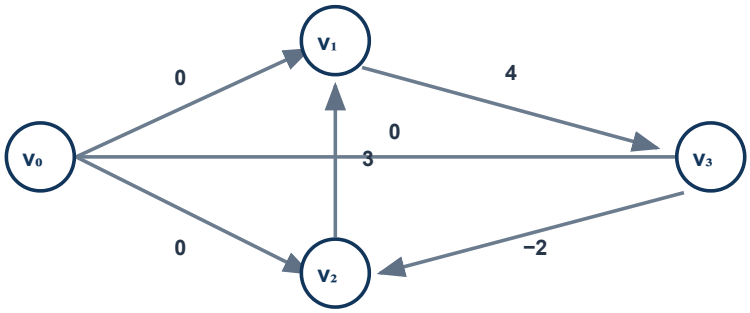

Figure 21.1 — The super-source ensures all constraint vertices are reachable; each inequality becomes one edge.

The theorem and proof intuition

If the constraint graph has no negative cycle, set $x_i = \delta(v_0, v_i)$. For every constraint edge $v_j \to v_i$ of weight b_k, triangle inequality gives $\delta(0,i) \le \delta(0,j) + b_k$, exactly $x_i - x_j \le b_k$.

If a negative cycle exists, summing its inequalities cancels all variables and yields $0 \le$ a negative number—impossible. Thus Bellman–Ford either returns a feasible assignment or certifies infeasibility.

22. Difference constraints — dry run, extensions, exam use

Worked feasible system

Constraint	Edge	With BF solution $(x_1,x_2,x_3)=(0,-2,0)$
$x_1-x_2\le 3$	$v_2\to v_1$ (3)	$0-(-2)=2\le 3$ ✓
$x_2-x_3\le -2$	$v_3\to v_2$ (−2)	$-2-0=-2\le -2$ ✓
$x_3-x_1\le 4$	$v_1\to v_3$ (4)	$0-0=0\le 4$ ✓

The super-source starts all distances at at most 0. Relaxing $v_3\to v_2$ lowers x_2 to -2; the remaining inequalities are satisfied.

Infeasible in one glance

$x_1-x_2\le 1$
$x_2-x_1\le -3$

Edges $v_2\to v_1(1)$, $v_1\to v_2(-3)$
Cycle weight $= -2 \Rightarrow$ infeasible.

Algebra agrees: the first says $x_1-x_2\le 1$, while the second says $x_1-x_2\ge 3$.

Useful facts and extensions

Fact	Exam-ready explanation
Translation invariance (Lemma 24.8)	If x is feasible, $x+d\cdot(1,1,\ldots,1)$ is feasible because differences do not change.
Complexity	With explicit v_0: $O(n(m+n))$. With the equivalent all-zero initialization and no v_0 edges: $O(nm)$.
Remove super-source	Initialize every $d[v]=0$ and relax only the m constraint edges; this is equivalent to zero edges from v_0.
Equality	$x_i=x_j+b$ becomes $x_i-x_j\le b$ and $x_j-x_i\le -b$.
Single-variable bound	Use a fixed $x_0=0$: $x_i\le b$ becomes $x_i-x_0\le b$.
Integer solutions	If all b values are integers and the system is feasible, BF distances are integers.

Maximum-mark construction: write the rearrangement, draw each edge in the correct direction j→i, add $v_0\to v_i(0)$, run BF, then explicitly substitute the returned distances into every inequality.

23. All shortest-path properties (CLRS 24.5)

Property	Precise statement	Why it matters
Triangle inequality	$\delta(s,v) \le \delta(s,u)+w(u,v)$ for every edge (u,v).	A shortest route to v is no worse than a shortest route to u followed by (u,v).
Upper bound	Always $d[v] \ge \delta(s,v)$; once equal, it never changes.	Relaxation stores weights of real discovered paths and only decreases them.
No path	If v is unreachable, $d[v]=\delta(s,v)=\infty$ forever.	No relaxation chain from s can reach it.
Post-relaxation	Immediately after RELAX(u,v), $d[v] \le d[u]+w(u,v)$.	The tested edge constraint is satisfied at that instant.
Convergence	If $s \to \dots \to u \to v$ is shortest and $d[u]=\delta(s,u)$, relaxing (u,v) makes $d[v]=\delta(s,v)$ forever.	Correctness propagates one edge.
Path relaxation	Relax edges of a shortest path in path order; its endpoint becomes correct, even with other relaxations interspersed.	Core proof tool for BF and DAG-SP.
Predecessor subgraph	When every d is correct and no reachable negative cycle exists, G_π is an SPT rooted at s.	Distances plus parents truly represent paths.

Why parent pointers do not form a cycle

Initially only s is in G_π. Suppose one relaxation first creates a predecessor cycle. Along old predecessor edges, each child's estimate was set from its parent and parent estimates could only fall later; the newly relaxed edge gives one strict improvement. Summing the inequalities around the cycle forces the cycle's total weight to be negative. That contradicts the assumption of no reachable negative cycle. Hence G_π stays a rooted tree.

> **Proof ladder:** triangle inequality → upper bound → convergence → path relaxation → algorithm distances → predecessor-subgraph property → actual shortest-path tree.

Subtle but examinable statements

- Relaxing an edge whose source estimate is ∞ makes no mathematical change; implementation still needs overflow protection.
- "$d[v] \le d[u]+w(u,v)$" is guaranteed immediately after relaxing (u,v), but a later decrease of d[u] can make that edge eligible again.
- Path-relaxation requires the chosen shortest-path edges to appear in order, not consecutively.
- Correct distances may admit several valid parent trees because shortest paths can tie.

24. Completeness map and lower-priority enrichment

CLRS Chapters 23–24 coverage map

Book topic	Where covered here	Depth
23.1 Growing an MST; generic method; safe/light edges; cut theorem	Sections 2–3	Full + proof
23.2 Kruskal, disjoint sets, Prim, priority queues	Sections 4–7	Full + dry runs
24 introduction: variants, subpaths, negative weights, cycles, path representation, relaxation	Sections 9, 13, 19–20	Full
24.1 Bellman–Ford	Sections 12–13	Full + proof
24.2 DAG shortest paths and PERT idea	Section 14	Full algorithm; application brief
24.3 Dijkstra	Sections 10–11	Full + counterexample + proof
24.4 Difference constraints	Sections 21–22	Complete core + extensions
24.5 Formal shortest-path properties	Section 23	All named properties

Lower-priority but useful chapter context

- **PERT / critical paths:** in a DAG, longest paths model earliest completion under precedence constraints. Negate weights and use DAG-SP, or replace min by max; cycles would invalidate the schedule model.
- **Reliability paths:** maximizing the product of edge reliabilities $r \in [0,1]$ becomes minimizing $\Sigma(-\lg r)$, producing nonnegative weights suitable for Dijkstra.
- **Arbitrage:** currency product >1 around a cycle becomes a negative cycle after weight $-\lg(\text{exchange rate})$; Bellman–Ford detects it.
- **MST sensitivity:** decrease a non-tree edge e; add e to T and, if beneficial, remove the maximum-weight edge on its tree path. This is the same exchange behind second-best MST.
- **MST verification:** for every non-tree edge (u,v), its weight must be at least the maximum edge on the T-path $u \to v$; otherwise an improving exchange exists.
- **Borůvka idea:** each component simultaneously chooses a cheapest outgoing edge, rapidly merging components. It is historically and theoretically important, but Kruskal/Prim are the standard exam algorithms here.

> **Boundary of "complete":** This note includes every teaching concept and named section in CLRS 23–24. The book's individual end-of-section exercises, research-history notes, and every specialized exercise variant are practice/enrichment rather than separate syllabus concepts.

Revision 1/5 — Definitions & properties

Graph core

- $G=(V,E)$; weighted edge $w(e)$.
- Tree = connected + acyclic.
- Forest = disjoint trees.
- Spanning tree: all V, exactly $V-1$ edges.
- Connected components: maximal connected subgraphs.

MST

Spanning tree minimizing $\Sigma w(e)$. Undirected connected input; negative weights allowed. Disconnected → MSF.

Cut: partition $(S, V-S)$. **Light:** minimum crossing edge. **Safe:** can join current A while A remains extendable to an MST.

Cut property: if cut respects A, a light crossing edge is safe. Proof: add it to an MST, delete the other cycle-crossing edge.

Cycle property: a strictly unique heaviest cycle edge is in no MST.

SSSP

$\delta(s,v)$=minimum path weight; ∞ if unreachable; $-\infty$ if a reachable negative cycle can lead to v.

```
INIT: d[s]=0; all other d=∞; parent=NIL
RELAX(u,v):
 if d[u]≠∞ and d[v]>d[u]+w(u,v):
   d[v]=d[u]+w(u,v); parent[v]=u
```

- $d[v]$ is an upper bound on $\delta(s,v)$.
- Triangle: $\delta(s,v) \le \delta(s,u)+w(u,v)$.
- Parents reconstruct path backward; reverse it.

Do not confuse

MST	SSSP
Min total tree weight	Min source→v path
No source	One source
Undirected	Usually directed

Uniqueness: all distinct weights ⇒ unique MST. Converse false. Equal weights may—but need not—create alternatives.

Revision 2/5 — MST algorithms

Kruskal: Sort–Check–Union

```
A=∅; MAKE-SET(v) for all v
sort E ascending
for (u,v) in E:
 if FIND(u)≠FIND(v):
   A += (u,v); UNION(u,v)
stop after V−1 accepted
```

- Forest grows; choice is global.
- DSU: path compression + union by rank.
- Time O(E lg V); DSU O(Eα(V)).
- Correct: component cut + light edge.

Second-best MST

$$w(T)+w(e)-\max \text{ edge on } T\text{-path}(u,v)$$

Try every non-tree e. Simple O(EV); LCA/max lifting O(E lg V). Strict version ignores equal-weight alternative MSTs.

Prim: Pull nearest branch

```
key[*]=∞; parent[*]=NIL; key[r]=0
Q=all vertices
while Q:
 u=EXTRACT-MIN(Q)
 for v in Adj[u]:
  if v∈Q and w(u,v)<key[v]:
   key[v]=w(u,v); parent[v]=u
```

- One tree grows; minimum frontier edge.
- Binary heap O(E lg V); array $O(V^2)$.
- Correct: extracted/unextracted cut.

Kruskal	Prim
Edges + DSU	Vertices + PQ
Sort required	No sort
Sparse/MSF	Dense with matrix

Traps: Prim key is a single edge, not root distance. Kruskal stops after V−1 *accepted*. Equal edges are legal. Root/tie choices may change the MST but not optimum weight.

Revision 3/5 — SSSP algorithms

Dijkstra (weights ≥0)

```
d[s]=0; PQ={(0,s)}
while PQ:
 (du,u)=extract-min
 if du≠d[u]: continue
 for (u,v):
  if d[v]>d[u]+w:
   update d[v],parent[v]; push
```

Greedy finalization. Binary heap O(E lg V); array $O(V^2)$. Proof: first wrong extracted vertex; nonnegative suffix cannot reduce path.

> One reachable negative edge is enough to invalidate the proof —even with no negative cycle.

DAG-SP (any weights)

```
topologically sort
for u in topo order:
 for (u,v): RELAX(u,v)
```

Θ(V+E). Negative weights okay; cycles impossible.

Bellman–Ford

```
initialize
repeat V−1 times:
 for every edge: RELAX
 if no change: stop
scan E once more:
 relaxable reachable edge ⇒ neg cycle
```

O(VE). After pass i, shortest paths of ≤i edges are captured. A simple shortest path uses ≤V−1 edges.

Negative cycle

- If reachable from s, Vth pass improves.
- Vertices reachable onward have no finite shortest path.
- Recover: improved x; follow parent V times; then loop to repeat.
- Ignore cycles unreachable from s.

Difference constraints: $x_i - x_j \le b$ maps to edge $j \to i(b)$. BF distances give a feasible assignment; a negative cycle means infeasible.

	Dijkstra	Bellman–Ford	DAG
Negative edge	No	Yes	Yes
Time	E lg V	VE	V+E
Idea	Greedy	Repeated relax	Topo order

Revision 4/5 — Proofs, formulas, complexities

Proof templates

MST exchange: Current A lies in some MST T. Add chosen light e, creating a cycle; remove another cut-crossing f. Since $w(e) \le w(f)$, new tree is no heavier and contains A+e.

Dijkstra: Suppose u first wrong extraction. On shortest path take first unsettled y and settled predecessor x. Relaxation makes $d[y]=\delta[y]$. Nonnegative suffix gives $\delta[y] \le \delta[u]$. Min-Q and upper-bound squeeze $d[u]=\delta[u]$.

Bellman–Ford: Induct on path-edge count. If prefix to u is correct after $i-1$ passes, relaxing (u,v) during pass i makes v correct.

DAG: Topological order processes every predecessor before its outgoing edge; one pass realizes recurrence.

Complexity wall

Algorithm	Time
Kruskal	E lg V
Prim heap / array	E lg V / V^2
Second MST simple / LCA	EV / E lg V
Dijkstra heap / array	E lg V / V^2
Bellman–Ford	VE
DAG-SP	V+E

Formulas

- Tree edges = $V-1$.
- $w(T)=\Sigma_{e \in T} w(e)$.
- Relax if $d[v] > d[u]+w(u,v)$.
- Second: $w(T)+w(e)-\text{maxPath}$.
- Matrix space V^2; list $V+E$.
- $\lg E = \Theta(\lg V)$ for simple connected graphs in MST analysis.

Revision 5/5 — Last-door checklist

Before a dry run

- □ Directed or undirected?
- □ Connected? DAG?
- □ Any negative reachable edge/cycle?
- □ Source/root and tie rule stated?
- □ Correct representation?

Show these columns

Algorithm	State to show
Kruskal	edge, accept/reject, components
Prim	extract, key[], parent[]
Dijkstra	settle, d[], parent[], PQ
Bellman	pass, d[], changed?

Answer skeleton

1. Definition + assumptions
2. Intuition
3. Pseudocode
4. Dry run / figure
5. Correctness invariant
6. Complexity with structure
7. Limitation/trap

Ten final traps

1. MST ≠ shortest-path tree.
2. Safe ≠ globally lightest.
3. Tied weights do not force multiple MSTs.
4. Prim key ≠ accumulated distance.
5. Visited on extraction, not discovery.
6. Dijkstra forbids reachable negative edges.
7. Bellman scans all edges $V-1$ times.
8. Vth relaxation must be reachable.
9. DAG may have negative edges, never cycles.
10. Parent path is backward; reverse it.

30-second algorithm picker
Undirected connect-all → MST.
DAG → topological SP.
General + negative → Bellman–Ford.
General + nonnegative → Dijkstra.
Difference inequalities → constraint graph + BF.

Memory line:
Kruskal sorts components together.
Prim grows one cheap branch.
Dijkstra finalizes cheap paths.
Bellman keeps correcting.
DAG obeys the order.

Final self-test

Can you state the cut property, write all four pseudocodes, explain $V-1$ in two different contexts (tree edges and Bellman passes), produce a negative-edge Dijkstra counterexample, and derive each complexity? If yes, walk in calmly.